1D Classical Repetition Code

Teresa del Aguila Ferrandis

January 2026

1 Decoding Algorithm

Step by step outline:

1. Define initial state ψ_i as an N-array of zeros (logical 0), and apply bit flip error to each element with p_i probability, creating the noisy state ψ_n.

2. Calculate syndromes using nearest neighbors as $s_i = 1$ if $\psi_{n(i)} \neq \psi_{n(i+1)}$ and $s_i = 0$ if $\psi_{n(i)} = \psi_{n(i+1)}$ and collect in an $N-1$ array.

3. Store the positions of errors/domain walls according to the syndrome in an array err_arr and identify if there are an even or odd number of errors N_{err}. An odd N_{err} means we have a bit flip at either the left or rightmost edge (i.e. something like $[1, 0, 0, ..., 0]$ or $[0, 0, 0, ..., 1]$).

4. In the **odd case** we reconstruct the noisy state as follows.

   - We will move in the left to right (forward) and right to left (backward) direction, pairing up elements the consecutive elements in the elements of err_arr. Since N_{err} is odd, one element will always be left unpaired and we will handle this one separately.
   - We propose which positions in our noisy state need bit flipping to recover the original state. For the error positions in err_arr that are paired up, say a and b form a pair, we define the positions in the noisy state that need bit flipping as those within the interval $(a, b]$ where we exclude a but include b.
   - Now for the unpaired indices. In the forward method, this will be the last element in err_arr, let me call it k for example, an we define the positions in the noisy state that need correction to be $(k, N]$. In the backwards method, this would be the first element of err_arr and instead we do define the positions to be corrected as $[0, k]$.
   - We will obtain a reconstructed noisy state from the forwards and backwards method, and I chose the lowest weight one (i.e. the one we the least non-zero elements, corresponding to least errors and least flips needed to correct state)
   - An example:

     noisy state = [0 1 0 0 1 0 0 1]
     syndrome = [1 1 0 1 1 0 1]
     error positions = [1 2 4 5 7]
     forwards method flips to implement: (1,2] (4,5] (7,8]
     backwards method flips to implement: [1,1] (2,4] (5,7]

forwards method predicts noisy state: [0 1 0 1 1 0 0 1] = ψ_f
backwards method predicts noisy state: [1 0 1 1 0 1 1 0] = ψ_b
lowest weight = ψ_f which is the correct reconstruction ✓

5. In the **even case** we reconstruct the noisy state as follows. The even case could mean we either have no bit flip errors at the edges or that we have two (i.e. something like $[1, 0, ..., 0, 1]$).

   - In this case we only carry out the forwards pairing up method, but we will still come up with two candidate states for the noisy state. For one of the candidate states, we will implement the bit flip corrections to an initial logical 0 state (i.e. a [0,0,0, ... ,0] state) and for the second candidate we will implement bit flip corrections to an initial logical 1 state (i.e. a [1,1,1, ... , 1] state.
   - We then once chose the lowest weight state.
   - An example would be:

     noisy state = [1 0 0 1 0 0 0 1]
     syndrome = [1 0 1 1 0 0 1]
     error positions = [1 3 4 7]
     forwards method flips to implement: (1,3] (4,7]
     backwards method flips to implement: [1,1] (2,4] (5,7]
     Initialized to 0 candidate predicts: [0 1 1 0 1 1 1 0] = ψ_0
     Initialized to 1 candidate predicts: [1 0 0 1 0 0 0 1] = ψ_b
     lowest weight = ψ_b which is the correct reconstruction ✓

This works most of the time, should look into tecnicalities of limitations, one set of notes mentioned "If the minimum distance is at least $2t + 1$, a nearest neighbor decoder will always decode correctly when there are t or fewer errors." Also need to figure out what to do when two proposed noisy states have the same weight, maybe majority vote method. Actually for most of these cases, the majority vote method would be enough to decode, but I guess the point was to use a syndrome-based approach instead, so that we are not directly using noisy state/directly counting data bits.

2 Link to Code

The code I used to test this algorithm for general cases can be found in this Github repository link.

3 Additional Example outputs:

These are all with a bit flip probability of 0.3.

```
Noisy state: [0 1 0 1 1 0 0 0 1 0]
Syndrome:  [1 1 1 0 1 0 0 1 1]
Error positions:  [1, 2, 3, 5, 8, 9]
flips in even case [2, 4, 5, 9]
from initialized zeros [0 1 0 1 1 0 0 0 1 0]
from initialized ones [1 0 1 0 0 1 1 1 0 1]
decoded state [0 0 0 0 0 0 0 0 0 0]
```

```
Noisy state: [0 0 0 0 1 0 0 1 0 0]
Syndrome:  [0 0 0 1 1 0 1 1 0]
Error positions:  [4, 5, 7, 8]
flips in even case [5, 8]
from initialized zeros [0 0 0 0 1 0 0 1 0 0]
from initialized ones [1 1 1 1 0 1 1 0 1 1]
decoded state [0 0 0 0 0 0 0 0 0 0]
```

```
Noisy state: [0 1 0 0 1 1 1 0 0 1]
Syndrome:  [1 1 0 1 0 0 1 0 1]
Error positions:  [1, 2, 4, 7, 9]
backwards interval: [ 7 9 ]
backwards interval: [ 2 4 ]
forward method flips suggested:  [2, 5, 6, 7, 10]
backwards method flips suggested:  [8, 9, 3, 4, 1]
noisy candidate f [0 1 0 0 1 1 1 0 0 1]
noisy candidate b [1 0 1 1 0 0 0 1 1 0]
we have reached an impasse...majority vote?
decoded state [0 1 0 0 1 1 1 0 0 1]
```

Example where it fails if I increase bit flip probability and get more errors.

```
Noisy state: [0 1 1 0 1 1 1 1 1 1]
Syndrome:  [1 0 1 1 0 0 0 0 0]
Error positions:  [1, 3, 4]
backwards interval: [ 3 4 ]
forward method flips suggested:  [2, 3, 5, 6, 7, 8, 9, 10]
backwards method flips suggested:  [4, 1]
noisy candidate f [0 1 1 0 1 1 1 1 1 1]
noisy candidate b [1 0 0 1 0 0 0 0 0 0]
decoded state [1 1 1 1 1 1 1 1 1 1]
```